

Modeling Agent Performance and Reviewer Interaction in Software Development: An Empirical Study

Abstract

The rise of large language models and autonomous coding agents has reshaped software development across tasks such as fixing bugs, testing code, building artifacts, and CI. Yet the strengths of different agents and their interaction with developers and reviewers across task types remain unclear. In this study, we analyze over 30,000 Pull Requests from the AIDEV-Pop dataset to examine whether accurate agents are also efficient. Our results showed that correlations varied across agents due to the inherent unpredictability of Resolution Time as a metric, but they also demonstrated that Merge Rate remains a stable and robust measure for empirical evaluation. We further evaluate whether substantive Code Churning correlates with extensive developer-agent interaction and observe a substantive positive relationship across different agents and task-types. These findings highlight the need for empirical, data-driven guidance on selecting and deploying coding agents in modern SE workflows and offer actionable insights for developers, researchers, and ecosystem maintainers.

Replication Kit: <https://github.com/VohraAK/AgentPerformance-RepKit>

1 Introduction

The invention of the transformer architecture [1] paved the way for rapid advances in Natural Language Processing (NLP) and enabled the development of Large Language Models (LLMs) [2] and autonomous coding agents [3]. Their prominence has driven a marked shift in software engineering practice: development workflows that once relied on developers manually coding, reviewing, and testing prototypes have transitioned toward intent-first, conversational development, with coding, testing, and fixes increasingly carried out by LLMs or specialized agents [4]. Recent studies report that agent-generated pull requests (PRs) now match or exceed human PR volume across many task types [5].

Despite this rise, significant challenges remain. Developers often rely on these agents without an empirical understanding on which models perform best in particular stages of the software development and maintenance cycle, how developer-agent interaction affects outcomes, and how agents should be deployed based on preferences for success, speed, or both. Moreover, although these agents now play a dominant role in the SE landscape, no prior work has systematically examined cases in which agents struggle, including high-interaction PRs and complex task types that require substantial back-and-forth between developers and agents. Understanding

the scope, success, and operational characteristics of agentic tasks is therefore essential. The AIDEV dataset provides the most extensive source of agentic PRs to date and enables quantitative evaluation and qualitative inspection of agent performance, reviewer interaction patterns, and task domains where agents exhibit difficulty.

The AIDEV-Pop subset includes PRs generated by five agents—OpenAI_Codex, ClaudeCode, Cursor, Copilot, and Devin—and captures metadata such as timestamps, task types, PR status, and interaction traces. Its scale offers a strong empirical foundation for analysing agent behaviour. While empirical studies on coding agents are emerging, existing work has generally focused on isolated SE activities such as refactoring performance [6] or code-review assistance. However, no existing research has conducted an end-to-end examination of agent performance across all major software engineering and maintenance stages, nor has prior work modelled developer/reviewer-agent interaction to identify the rationale behind agent modifications and updates or derive actionable practices. To address this gap, we study:

RQ1: Does Merge Rate (success & accuracy) correlate with Resolution Time (speed & efficiency) for individual PRs across different tasks?

RQ2: How does the nature of agent-developer interactions impact Code Churning?

To answer these questions, we analyze approximately 32,000 agentic PRs in AIDEV-Pop, computing Merge Rates, Resolution Times, Code Churn (changes in code), and Comment Volume (number of review comments per PR) for five agents across twelve distinct task types. We conduct correlation analyses to study relationships between agent performance: Merge Rate and Mean Resolution Time, and interaction behavior: Code Churn and Comment Volume. For replication purposes, we publicly release a replication package containing all scripts, queries, and derived data. The subsequent sections describe the dataset and methodology, present detailed findings for each research question, examine related studies, and discuss threats to validity and the final section outlines implications for developers, researchers, and the wider SE ecosystem.

2 Dataset Description

For this study we use the AIDEV dataset [7], a large-scale collection of nearly 1,000,000 mined pull requests (PRs) originating from both human developers and autonomous coding agents. AIDEV is publicly available through Hugging Face and contains detailed metadata for each PR, including timestamps, user identifiers, code changes, and review interactions. From this source, we focus on AIDEV-Pop, a curated subset containing only agent-generated PRs that have been topic-modelled [8][9] into task types representing different stages of the software development and maintenance life-cycle. Each entry in AIDEV-Pop corresponds to a single pull request created by one of several coding agents.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR 2026, Rio de Janeiro, Brazil

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Key fields include:

- **task_type**: A categorical label representing the inferred software engineering activity associated with the PR. These include development and maintenance tasks such as *build*, *test*, *fix*, *refactor*, *chore*, and others. These task types are derived from topic modelling applied to the original dataset of mined PRs.
- **agent**: The name of the coding agent responsible for generating the PR (OpenAI_Codex, ClaudeCode, Cursor, Copilot, or Devin).
- **created_at**, **merged_at**, **closed_at**: Timestamps indicating when the PR was opened, merged, or closed, enabling computation of temporal metrics such as resolution time and filtering of active versus completed PRs.
- **merged (boolean)**: Indicates whether the PR was successfully incorporated into the target repository.

In addition to the main AIDEV-Pop dataframe, the dataset includes a collection of JSON files containing detailed review and code-modification information for each PR. From these JSON objects, we extract:

- **PR_Id**: A unique identifier used to link PRs between the tabular dataset and the comment/code-change dataset.
- **additions and deletions**: Counts of lines of code added and removed by the PR, used to compute *code churn*.
- **comment_body**: The textual content of each review comment.
- **user_type**: Identifies whether a comment was made by a human reviewer or by the agent itself, enabling measurement of *Comment Volume*.
- **agent**: Specifies which model produced the PR or participated in review responses.

By parsing these JSON objects into a structured CSV, we obtain a companion dataset that aligns with AIDEV-Pop through shared PR identifiers. Together, these two components provide a unified view of agent activity across repositories, encompassing task type, temporal characteristics, code modifications, and review interactions. This combined dataset allows us to compute the four key metrics central to our study: merge rate, mean resolution time, code churn, and Comment Volume. These metrics enable a quantitative and qualitative examination of agent performance for RQ1 along with interaction behaviour for RQ2. The structured nature and coverage of AIDEV-Pop make it suitable for analyzing agent variability across task types and for investigating the broader dynamics of agent participation in modern software development workflows.

3 Methodology

To address our two research questions, we combined data extraction, preprocessing, statistical analysis, and qualitative inspection using the AIDEV-Pop dataset. The total number of PRs analyzed is $\approx 32,000$ across 11 task-types for 5 coding agents: Claude-Code, Devin, Cursor, Copilot and OpenAI_Codex.

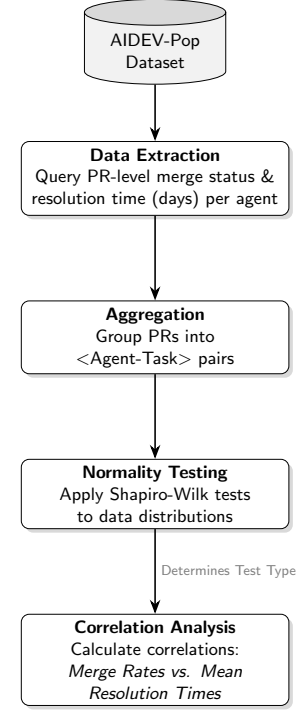


Figure 1: RQ1 Methodology flowchart: Merge Rate and MRT

RQ1: Merge Rate and Mean Resolution Time

We first loaded the AIDEV-Pop environment and used NumPy and associated Python routines to extract all agent-generated pull requests into a unified CSV file. Since merge and resolution time computations require completed PRs, we removed all open PRs, identified by missing `closed_at` timestamps. This yielded a filtered subset of closed PRs across all agents. Next, we queried this subset to generate per-agent dataframes partitioned by `task_type`. These task types originate from topic modelling performed on the original mined dataset and were made available directly in the `AIDEV_pop` fields. For each agent and task type, we computed merge rate as the ratio of PRs containing a `merged_at` timestamp to the total number of closed PRs. Resolution times were computed by subtracting `opened_at` from `merged_at` on a per-PR basis, after which we computed the MRT per agent-task grouping. We then tested the relationship between merge rate and mean resolution time. The Shapiro-Wilk test was applied to each variable to assess normality, and both were found to be non-parametric. Accordingly, we applied Spearman’s rank correlation across agent-task pairs. Spearman’s correlation coefficient is defined as:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Where:

- (1) d_i is the difference between ranked pairs
- (2) n denotes the number of observations

RQ2: Code Churn and Comment Volume

For RQ2, we examined the relationship between Code Churn and Comment Volume. We parsed the JSON objects stored in the AIDEV directory, extracting comment bodies, user types, timestamps, and modification metadata such as additive and subtractive line changes. These were processed into a structured CSV through custom Python scripts. Code Churn for each PR was computed as the difference between additions and deletions. Comment Volume was defined as the number of review comments / PR, derived from filtering comment_body entries where the user_type indicated a human reviewer or agent response. As in RQ1, we aggregated these values by agent and task type, yielding average churn and average Comment Volume for each agent-task pair. We again used the Shapiro-Wilk test to assess normality and found both distributions to be non-parametric, leading us to apply Spearman's rank correlation between average churn and Comment Volume.

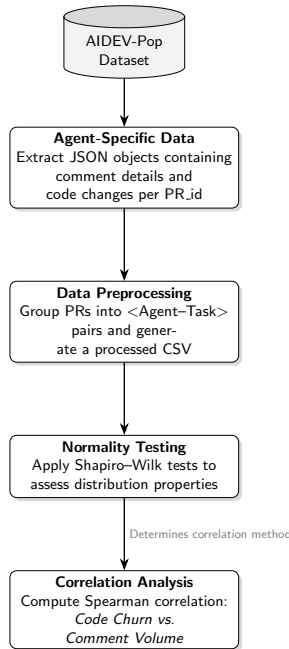


Figure 2: RQ2 Methodology flowchart: Code Churn and Comment Volume

4 Analysis and Findings

4.1 RQ1:

Our results show that these correlations are **agent specific**. Out of all of the agent-task pairs, 69% were negatively-correlated. This suggests that for most agents, particularly Devin, PRs that are merged in greater density tend to be resolved much faster than their rejected counterparts. For instance, in (chore) tasks, Devin demonstrates a moderate negative correlation ($r = -0.35$), implying that correct PRs are often accepted immediately, while problematic ones may not be merged at all.

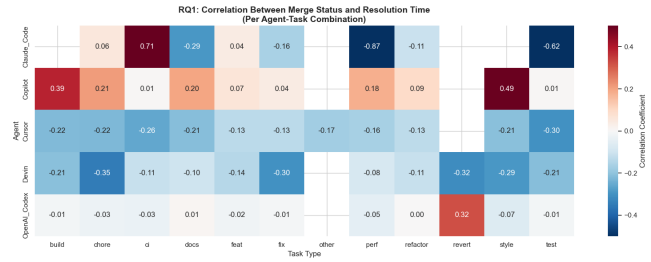


Figure 3: Correlation Heatmap of Merge Status vs. Resolution Time.

In contrast, **Claude_Code** shows the opposite trend for Continuous Integration (CI) tasks ($r = +0.71$), where successfully merged PRs actually take longer to resolve than rejected ones. This implies that for Claude_Code, higher quality work often involves more time for review and discussion. Meanwhile, **OpenAI_CoDeX**, which covers the vast majority of our dataset, shows showcases no significant link between accuracy and resolution speed (except for in *revert*) where a moderate value of 0.32 indicates that at large scales, resolution time is not a reliable predictor of whether or not a PR will be accepted. It should be noted that a negative correlation here indicates that more prolific agents also tend to have their requests resolved in a timely manner, reflected by a lower MRT value. The variance in correlations can be attributed to the inherent unpredictability of resolution time, where factors such as developer trust, reviewer activity, and agent inference jointly influence how promptly a pull request is merged. Accordingly, agents with higher merge rates may be more reliable choices for particular task types, as merge outcomes provide a more tangible and less externally influenced indicator of agent performance than resolution time.

Answer to RQ1

The relationship between accuracy and resolution speed depends on the agent. Each agent may have a different correlation between these variables, depending on the task types. Developers should prioritize Merge Rate data to obtain more reliable empirical insights.

4.2 RQ2:

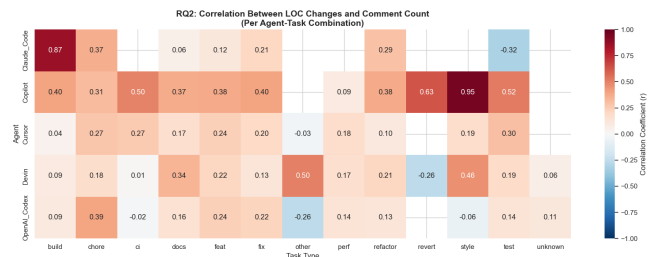


Figure 4: Correlation Heatmap of Code Churn vs. Comment Volume

We investigate whether larger code changes are linked to more intense reviewer-agent interaction by analyzing the correlation between code churn and comment volume for each agent-task combination. Using PR-level data and Spearman’s correlation, we evaluate 58 valid combinations across all five agents. Figure 4 summarizes the distribution of correlation coefficients. Overall, 82.8% of correlations are positive, indicating a consistent tendency for larger PRs to attract more discussion. The average absolute correlation is 0.258, reflecting a small to moderate effect. Copilot shows the strongest and most stable pattern (mean $r = 0.449$), suggesting that its larger PRs reliably require more reviewer engagement. Devin, Cursor, and OpenAI_Codex exhibit weaker but still predominantly positive relationships, while Claude_Code shows higher variability due to limited sample sizes. Across task types, *feat*, *fix*, and *docs* consistently produce positive correlations for multiple agents, aligning with the inherently review-intensive nature of these tasks. Other task types display more variability, indicating that discussion volume is influenced by factors beyond code churn alone. These findings suggest that while code modification is not a dominant predictor of the nature of an reviewer-agent interaction, it remains a dependable indicator of expected review effort in most agent-task contexts.

Answer to RQ2

PRs with larger code modifications generally receive more review comments. The strength of this relationship varies by agent and task type, with Copilot showing the most consistent patterns and Cursor and Codex the weakest.

5 Threats to Validity

While our analysis offers valuable insights, several potential threats affect the robustness of our findings. First, although we examine more than 30,000 agent-generated PRs across five agents and twelve task types, the dataset does not fully represent the broader software development landscape. The emergence of multi-agent conversational tools, such as Anti-Gravity, suggests that agent behaviour now extends beyond what is captured in AIDEV_pop, limiting the generalizability of our results. Sampling bias also arises from the distribution of PRs: approximately 20,000 originate from Codex, while the remaining PRs are split among Claude_Code, Cursor, Copilot, and Devin. This imbalance influences certain observed patterns, although it reflects usage frequencies in the mined dataset. Another threat concerns the calculation of Mean Resolution time (MRT), as mentioned earlier. Although MRT is derived directly from timestamps, it is sensitive to external factors such as reviewer availability, developer trust, and repository activity, which introduce noise into the measurements. For RQ2, additional limitations stem from parsing review comments from JSON objects; inconsistencies in message bodies restricted our ability to extract friction data uniformly across all PRs, resulting in a subset that may not fully represent the complete dataset.

Future work may mitigate these threats by incorporating more balanced and dynamic datasets, expanding to multi-agent development environments, and improving comment-parsing reliability to strengthen the validity of interaction-level analysis.

6 Related Work

Recent studies have examined the use of large language models and automated coding tools in software engineering, including code generation, program repair, and conversational development workflows [10] [11] [12]. Work on human-AI collaboration further explores developer trust, review behaviour, and the reliability of model-generated code [13]. Empirical research on pull requests has analysed merge success factors, resolution times, code churn effects, and reviewer-author interaction patterns [14] [15]. These studies establish key behavioural dynamics that inform our analysis of agent-generated PRs. Topic-modelling approaches have also been used to classify software tasks and categorize PRs, aligning with the task-type structure in AIDEV-Pop [9]. Comparative evaluations of coding assistants such as GitHub Copilot, OpenAI Codex, Claude-Code, and similar models have assessed code quality, correctness, refactoring ability, and behavioural differences across models [6]. Additional work highlights limitations and failure modes of LLMs, including hallucinated code and high-modification patches [16]. Finally, several MSR studies have focused on mining large datasets of AI-generated code, PR behaviour, and automated development patterns, motivating the use of AIDEV-Pop for ecosystem-scale analysis [7]. However, prior work has not jointly examined agent accuracy, efficiency, and reviewer interaction across multiple agents and task types. Our study addresses this gap through a combined analysis of performance and interaction metrics for agentic PRs.

7 Conclusion

Our analysis reveals that there is no universal standard for agent efficiency; instead, each agent works differently. For Devin, faster PRs are merged more often, as seen in chore and fix tasks. Copilot flips this script, where successful merges often take longer to resolve, specifically in style and build tasks, pointing to a more iterative review cycle. OpenAI_Codex, however, remains scale-independent, shows virtually no link between speed and success with near-zero correlations across the board. On the human interaction side, larger code changes generally spark more discussion, but Copilot stands out as the most predictable collaborator. It shows a massive correlation between code size and comment volume in style and test tasks, meaning developers should expect proportional review burdens. Other agents like Cursor and OpenAI_Codex show much looser connections, suggesting their discussions are driven by complexity or context rather than raw size. Ultimately, teams should select Devin for rapid, autonomous fixes and Copilot when they are ready for a thorough, collaborative engineering workflow.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [3] X. Qian, C. Cong, W. Yang, Y. Chen, J. Su, Z. Xu, M. Liu, and M. Sun, "Swe-agent: Autonomous software engineering agents for multi-step computer use," *arXiv preprint arXiv:2307.02953*, 2023.
- [4] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "Codet: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2022.
- [5] M. Watanabe *et al.*, "On the use of agentic coding: An empirical study of pull requests on github," *arXiv preprint arXiv:2509.14745*, 2025.
- [6] K. Horikawa *et al.*, "Agentic refactoring: An empirical study of ai coding agents," *arXiv preprint arXiv:2511.04824*, 2025.
- [7] H. Li, H. Zhang, and A. E. Hassan, "The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering," *arXiv preprint arXiv:2507.15003*, 2025.
- [8] C. C. Silva, M. Galster, and F. Gilson, "Topic modeling in software engineering research," *Empirical Software Engineering*, vol. 26, no. 6, p. 120, 2021.
- [9] A. Panichella *et al.*, "How to effectively use topic models for software engineering tasks? an approach based on genetic algorithms," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013.
- [10] Q. Zhang *et al.*, "A systematic literature review on large language models for automated program repair," *arXiv preprint arXiv:2405.01466*, 2024.
- [11] O. Cico, B. Cico, and A. Cico, "Ai-assisted software engineering: a tertiary study," in *2023 12th Mediterranean Conference on Embedded Computing (MECO)*. IEEE, 2023.
- [12] S. I. Ross *et al.*, "The programmer's assistant: Conversational interaction with a large language model for software development," in *Proceedings of the 28th International Conference on Intelligent User Interfaces*, 2023.
- [13] A. Alami and N. Ernst, "Human and machine: How software engineers perceive and engage with ai-assisted code reviews compared to their peers," in *2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 2025.
- [14] G. Kudrjavets, N. Nagappan, and A. Rastogi, "Do small code changes merge faster? a multi-language empirical investigation," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022.
- [15] Y. Yu, H. Wang, G. Yin, and T. Wang, "Reviewer recommendation for pull-requests in github: What can we learn from code review and bug assignment?" *Information and software technology*, vol. 74, pp. 204–218, 2016.
- [16] F. Liu *et al.*, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv preprint arXiv:2404.00971*, 2024.